

作业：API 设计与实现

作业目标

完成项目的 RESTful API 设计，并实现前后端的基本对接，建立协作契约。

具体要求

1. API 规范设计

遵循 RESTful 设计原则：

- 使用名词复数作为资源路径（如 `/users`, `/orders`）
- 使用 HTTP 方法表示操作（GET/POST/PUT/DELETE）
- 使用合适的状态码（200/201/400/401/404/500）
- 统一的响应格式（JSON）

2. API 文档编写

使用 OpenAPI (Swagger) 规范编写 API 文档：

```
openapi: 3.0.0
info:
  title: 项目名称 API
  version: 1.0.0
paths:
  /api/users:
    get:
      summary: 获取用户列表
      responses:
        '200':
          description: 成功
    post:
      summary: 创建用户
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'
```

3. 核心 API 列表

至少设计以下 API：

- 用户认证（注册、登录、登出）

- 核心业务 CRUD（至少一个资源）
- 数据查询（分页、筛选）

4. 前端 API 访问层实现

根据选定的前端技术栈实现 API 调用：

Web (JavaScript/TypeScript):

```
// src/api/client.ts
const apiClient = axios.create({
  baseURL: '/api',
  headers: { 'Content-Type': 'application/json' }
});

// 请求拦截器：添加 Token
apiClient.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Android (Kotlin):

```
// api/ToDoApi.kt
interface ToDoApi {
  @GET("todos")
  suspend fun getTodos(): Response<ToDoListResponse>

  @POST("todos")
  suspend fun createToDo(@Body todo: ToDoRequest): Response<ToDoResponse>
}
```

iOS (Swift):

```
// Services/APIClient.swift
class APIClient {
  func getTodos() async throws -> [ToDo] {
    // 实现 API 调用
  }
}
```

5. 后端 API 端点实现

根据选定的后端技术栈实现 API：

Python (FastAPI):

```
@app.get("/api/todos")
async def list_todos(page: int = 1, size: int = 10):
    todos = await db.todos.find_all(page, size)
    return {"code": 200, "data": todos}

@app.post("/api/todos", status_code=201)
async def create_todo(todo: TodoCreate):
    new_todo = await db.todos.create(todo.dict())
    return {"code": 201, "data": new_todo}
```

Node.js (Express):

```
router.get('/api/todos', async (req, res) => {
    const todos = await Todo.find();
    res.json({ code: 200, data: todos });
});

router.post('/api/todos', async (req, res) => {
    const todo = await Todo.create(req.body);
    res.status(201).json({ code: 201, data: todo });
});
```

6. Mock 数据配置

在后端未完成时，使用 Mock 数据：

方式一：JSON Server

```
// db.json
{
  "todos": [
    { "id": 1, "title": "示例任务", "completed": false }
  ]
}
```

```
json-server --watch db.json --port 3001
```

方式二：MSW (Mock Service Worker)

```
// mocks/handlers.ts
export const handlers = [
  rest.get('/api/todos', (req, res, ctx) => {
    return res(ctx.json({ code: 200, data: mockTodos }));
  })
];
```

7. API 测试

使用 Postman 或 Apifox 创建测试集合：

- 导入 OpenAPI 文档
- 配置环境变量
- 编写测试用例（至少 5 个）

后端单元测试示例：

```
# tests/test_api.py
def test_create_todo():
    response = client.post("/api/todos", json={"title": "Test"})
    assert response.status_code == 201
    assert response.json()["data"]["title"] == "Test"

def test_list_todos():
    response = client.get("/api/todos")
    assert response.status_code == 200
```

目录结构

```
project/
├── docs/
│   ├── api.yaml           # OpenAPI 规范文档
│   ├── api.md             # API 使用说明
│   └── contributions/     # 个人贡献说明
│       └── 04-api/
│           ├── zhangsan.md
│           └── lisi.md
├── frontend/              # 前端项目
│   └── src/
│       ├── api/           # API 访问层
│       │   ├── client.ts  # HTTP 客户端配置
│       │   └── todos.ts   # Todo API 调用
│       └── mocks/         # Mock 数据（可选）
│           └── handlers.ts
```

```
├─ backend/                # 后端项目（如有）
│   └─ app/
│       └─ routes/        # API 路由
│       └─ tests/         # API 测试
├─ db.json                # JSON Server 数据（可选）
└─ README.md
```

提交内容

Git 仓库提交

1. docs/api.yaml - OpenAPI 规范文档
2. docs/api.md - API 使用说明
3. 前端 API 访问层代码（frontend/src/api/）
4. 后端 API 实现（backend/app/routes/）或 Mock 配置
5. docs/contributions/04-api/你的名字.md - 个人贡献说明

学习通提交

提交以下截图作为个人进度证明：

1. Git 提交记录截图 — `git log --author="YOUR_NAME" --oneline --graph` 终端截图
2. Apifox/Postman 测试截图 — 测试集合运行成功的截图
3. 前端调用 API 截图 — 浏览器控制台或应用界面显示 API 数据
4. 后端 API 响应截图 — 终端显示 API 请求日志或测试通过

💡 截图需能证明你的实际贡献

提交截止

下次课前

个人贡献说明

每位成员在 docs/contributions/04-api/ 下提交个人贡献说明：

文件名：你的名字.md（如 zhangsan.md）

内容模板：

```
# API 设计与实现贡献说明
```

姓名：XXX

学号：XXX

日期：XXXX-XX-XX

我完成的工作

1. API 设计

- [] 用户认证 API
- [] 业务资源 API
- [] 查询接口设计

2. 文档编写

- [] OpenAPI 文档
- [] API 使用说明

3. 前端实现

- [] HTTP 客户端配置
- [] API 调用函数封装
- [] Mock 数据配置

4. 后端实现

- [] API 路由定义
- [] 业务逻辑处理
- [] 错误处理

5. 测试

- [] Postman/Apifox 测试集合
- [] 后端单元测试
- [] 测试用例数量: X 个

PR 链接

- PR #X: <https://github.com/xxx/xxx/pull/X>

遇到的问题和解决

1. 问题: XXX
解决: XXX

心得体会

(简要说明在 API 设计与实现过程中的收获)

评分标准

项目	分值	说明
RESTful 规范	15%	路径、方法、状态码使用正确
API 文档完整	15%	OpenAPI 规范，描述清晰
前端 API 层	20%	正确封装 API 调用，处理认证
后端/Mock 实现	20%	API 可正常访问，返回正确格式
测试用例	15%	覆盖核心接口
个人贡献说明	15%	贡献清晰，有截图证明

额外加分项

- 完整的前后端联调（而非仅 Mock）
- 后端实现单元测试
- 使用 MSW 或类似方案实现前端 Mock
- API 错误处理完善（网络错误、超时、重试）

参考资源

- OpenAPI 规范
- RESTful API 设计指南
- Apifox 使用教程
- Mock Service Worker
- JSON Server